Qualité Logicielle

Vérification déductive de programmes - Part 1

Asma BERRIRI

asma.berriri@universite-paris-saclay.fr

Page web du cours :
https://sites.google.com/view/asmaberriri/home/app5modelisation-verification

Outline

1. Vérification déductive de programmes
   - Design by contract – Programmation par contrats
   - Vérification déductive – Logique de Hoare

Contrats/Spécification

- But : Spécification des programmes/procédures/fonctions
- Approche : donner des assertions (i.e. propriétés) sur la procédure

Contrat

- Précondition
  - doit être vraie à l'entrée
  - assurée par l'appelant
- Postcondition
  - doit être vraie à la sortie de la procédure
  - assurée par la procédure elle-même si elle termine
  - aucune garantie si la précondition n'est pas satisfaite
  - terminaison peut être garantie ou non

Sémantique

Si la fonction est appelée dans un état satisfaisant la précondition, elle garantit la postcondition à la fin.

Exemple informel de contrat

```
/**
 * @brief Trie les valeurs pointées par ptr1, ptr2 et ptr3 dans l'ordre
       croissant, et retourne leur produit.
 * @pre ptr1, ptr2, ptr3 doivent être des pointeurs valides.
 * @pre Le produit ne doit pas provoquer d'overflow.
 * @return Le produit de *ptr1, *ptr2, *ptr3.
 */
int reorder(int* ptr1, int* ptr2, int* ptr3);
```

- Spécification claire, mais encore informelle (comment vérifier le tri ?).
- Idéal pour documentation, mais insuffisant pour la vérification formelle.

- Un bon contrat doit :
  - ↪ préciser clairement les **conditions d'appel valides**.
  - ↪ énoncer ce qui est **garanti en sortie**.
  - ↪ décrire les effets (sur pointeurs, tableaux, etc.).
- Il doit être :
  - ↪ **correct** (pas trop fort — sinon invalide),
  - ↪ **utile** (pas trop faible — sinon sans intérêt).

Contrats pour procédures

$$\text{Precondition(State)} \implies \text{Postcondition(procedure(State))}$$

- Etat?: snapshot du système, à chaque point de programme durant l'exécution (En C: les valeurs des variables et des paramètres)
- Precondition et Postcondition
  - Formules de la logique du 1er ordre
  - Utilisent les variables globales et les paramètres de la fonction comme variables libres[a]

[a] Soient x une variable et A une formule. Dans une formule $\forall x\ A$ ou $\exists\ xA$, la portée de la liaison pour x est A. Une occurrence de x dans une formule est libre si elle n'est pas dans la portée d'une liaison pour x, sinon elle est dite liée.

- Exemples

$$x = 3$$

$$y > x$$

$$(x \neq 0) \implies (y + z = w)$$

$$s = (\sum_i a[i] \mid 0 \leq i < N$$

$$\forall i \in 1..n | a[i] > a[i-1]$$

$$vrai$$

Plus faible precondition/ Plus forte postcondition

- La plus faible précondition

  i.e. une précondition impliquée par toutes les autres $\Rightarrow$ le plus large domaine de définition pour la fonction

  - $\{x = 5 \wedge y = 10\}\ z := x/y\ \{z < 1\}$
  - $\{x < y \wedge y > 0\}\ z := x/y\ \{z < 1\}$
  - $\{y \neq 0 \wedge x/y < 1\}\ z := x/y\ \{z < 1\}$

  Toutes sont vraies, mais la dernière est la plus utile car elle nous permet d'invoquer le programme dans l'état le plus général

- La plus forte postcondition

  i.e. une postcondition qui implique toutes les autres $\Rightarrow$ une postcondition la plus complète

  - $\{x = 5\}\ x := x * 2\ \{true\}$
  - $\{x = 5\}\ x := x * 2\ \{x > 0\}$
  - $\{x = 5\}\ x := x * 2\ \{x = 10 \vee x = 5\}$
  - $\{x = 5\}\ x := x * 2\ \{x = 10\}$

  Toutes sont vraies, mais la dernière est le plus utile

Spécifications formelles en ACSL

- **ACSL (ANSI C Specification Language)**: langage d'annotations pour le C
- Formuler des propriétés logiques vérifiables automatiquement

```
/*  @ requires  len >= 0 ∧ array.length = len
    @ ensures  \result == (\sum int j; 0 <= j ∧ j < len; array[j])
    @*/
int sum(int array[], int len) {
    int sum = 0;
    int i = 0;
    while (i < len) {
        sum = sum + array[i];
        i = i + 1;
    }
    return sum;
}
```

- Les types sont garantis par le compilateur : $(len, sum, i, \ldots :$ integer$)$
- Spécification
  - Precondition : $len \geq 0 \wedge array.length = len$
  - Postcondition : $(\sum_j array[j] \mid j \in \mathbb{N} \mid 0 \leq j \wedge j < len)$

Sémantique d'un contrat ACSL

- **requires** : précondition (contexte valide à l'entrée).
- **ensures** : postcondition (garantie de sortie si la pré est satisfaite).
- Les quantificateurs logiques (*forall*, *exists*, *sum*) permettent des propriétés globales.
- On peut aussi référencer :
  - l'état mémoire à l'entrée : *old*(...)
  - des plages mémoire valides : *valid*(...)
- Exemples de contrats ACSL

```
/*@ requires a >= 0;
    ensures \result * \result <= a < (\result + 1)*(\result + 1);
        */
int root(int a) { ... }

/*@ requires \valid(t+(0..n-1)) && n > 0;
    ensures \forall integer i; 0 <= i < n ==> \result <= t[i];
    ensures \exists integer k; 0 <= k < n && \result == t[k]; */
int getMin(int t[], int n) { ... }
```

2. Vérification déductive: Logique de Hoare

- Floyd (1967), Hoare(1969), Dijkstra (1978)
- Pour les langages impératifs
- Sémantique orientée vers la preuve de programmes
- Avec cette méthode sémantique, un programme est vu comme un transformateur de propriétés logiques

Hoare logic

- Un formalisme permettant de relier l'état initial et l'état final d'un programme
- Utilise des triplets de correction partielle pour spécifier et raisonner sur le comportement des programmes: $\{P\}\ C\ \{Q\}$
- Ici C est un programme et P et Q sont des prédicats d'état (des assertions logiques: spécification du programme /contrat)
- Assertions logiques : formules du premier ordre, avec comme formules atomiques les expressions du langage de programmation
  - P est la précondition et décrit l'état initial
  - Q est la postcondition et décrit l'état terminal (final)

Sémantique

- $\{P\}\ C\ \{Q\}$ est valide si et seulement si: lorsque le programme C est exécuté dans n'importe quel état satisfaisant P, alors, si l'exécution se termine, son état résultant satisfait Q
- Correction partielle : uniquement le comportement prévu en cas de fin d'exécution est spécifié

Sémantique des triplets : un peu plus formellement

- Un état (mémoire) d'un programme associe une valeur à chaque variable du programme. Un état peut être vu comme une fonction des variables vers les valeurs (ici entières)

```
z := 0;
x := 1;
y := 2;          (1)
z := x + y;      (2)
```

Au point de programme (1), l'état σ_1 est $z \leftarrow 0; x \leftarrow 1; y \leftarrow 2$

Au point de programme (2), l'état σ_2 est $z \leftarrow 3; x \leftarrow 1; y \leftarrow 2$

- Soit σ un état mémoire, soit P une assertion logique, on dit que σ satisfait P quand l'interprétation de P est vraie lorsque les variables libres[1] de P ont leur valeur courante (dans l'état). On écrit alors $\sigma \models P$

$\hookrightarrow$ σ_1 ne satisfait pas l'assertion $z \geq x$ (qui est fausse en 1)

$\hookrightarrow$ σ_2 satisfait l'assertion $z \geq x$ (qui est vraie en 2)

[1] Soient x une variable et A une formule. Dans une formule $\forall x\ A$ ou $\exists\ xA$, la portée de la liaison pour x est A. Une occurrence de x dans une formule est libre si elle n'est pas dans la portée d'une liaison pour x, sinon elle est dite liée.

Sémantique des triplets : un peu plus formellement

$\{P\}\ C\ \{Q\}$ est valide (on note $\models \{P\}\ C\ \{Q\}$) ssi pour tout état initial σ qui valide P ($\sigma \models P$), si l'exécution de C dans σ termine, soit σ' l'état final résultant, alors σ' satisfait Q ($\sigma' \models Q$)

- Quelques triplets de Hoare valides

$$\models \{true\}\ x := 5\ \{x = 5\}$$
$$\models \{x = y\}\ x := x + 3\ \{x = y + 3\}$$
$$\models \{x > 0\}\ x := x * 2\ \{x > -2\}$$
$$\models \{x = a\}\ \textit{if}\ (x < 0)\ \textit{then}\ x := -x\ \textit{else skip}\ \{x = abs(a)\}$$

- Mais ces triplets ne sont pas valides

$$\{x < 0\}\ x := x - 3\ \{x > 0\}$$
$$\{i = 1\}\ \textit{while}\ (i <= 10)\ \textit{do}\ i := i + 1\ \textit{done}\ \{i = 10\}$$

Hoare logic

- Pour définir une logique Hoare
  - ↪ Le langage de programmation sur lequel nous voulons raisonner, ainsi que sa sémantique opérationnelle
  - ↪ Un langage d'assertion pour définir les prédicats d'état, ainsi qu'une sémantique
  - ↪ Une interprétation formelle des triples de Hoare, ainsi qu'un Système formel de preuve pour dériver les triplets de Hoare

POLYTECH PARIS-SACLAY

Syntaxe d'un langage impératif

Version simplifiée d'un langage de programmation	programme = corps d'une méthode pas d'invocation types booléens, entiers et tableaux expressions arithmétiques et logiques de base affectations, séquencement, conditionnel, boucle while

- Expressions

$$
\begin{array}{rll}
e ::= & x & \text{variable} \\
| & z & \text{constante entière} \\
| & e + e \mid -e \mid e * e \mid e/e & \text{expressions arithmétiques}
\end{array}
$$

- Conditions (expressions booléennes)

$$b \ ::= \ true \mid false \mid e = e \mid \ e \leq e \mid !e \mid (e) \mid \ \ldots$$

- Instructions/commande

$$
\begin{array}{rll}
c ::= & x := e & \text{affectation} \\
| & \text{if } b \text{ then } c \text{ else } c & \text{conditionnelle} \\
| & \text{while } b \text{ do } c & \text{boucle} \\
| & c \ ; \ c & \text{séquence} \\
| & skip & \text{sans effet}
\end{array}
$$

- Les états sont des mappages de variables vers des nombres naturels

Le langage des assertions

- Ici le langage des prédicats (logique du 1er ordre)

$$P \ ::= \ b \mid P \wedge P \mid P \vee P \mid \ \neg P \mid P \implies P \mid \forall P \mid \exists P$$

- Toutes les expressions booléennes (b) du langage impératif sont des assertions
- Mélange de variables du programme et de variables logiques
- Implicitement les variables prennent des valeurs entières

↪ On dispose d'une sémantique pour ce langage d'assertions

- Les prédicats d'état P et Q peuvent faire référence à des variables de programme et seront écrits en utilisant des notations mathématiques standard et des opérateurs logiques tels que :

$$\wedge\ (and),\ \ \vee\ (or),\ \ \neg\ (not),\ \ \implies\ \ (implies)$$

- Par exemple, le prédicat $X = Y + 1 \wedge Y > 0$ décrit les états dans lesquels la variable Y contient une valeur positive et la valeur de X est égale à la valeur de Y plus 1.

Triplet de Correction partielle

- Le triplet de correction partielle $\{P\}\ C\ \{Q\}$ est valide si et seulement si
  - ↪ chaque fois que C est exécuté dans un état initial satisfaisant P
  - ↪ et cette exécution termine
  - ↪ alors l'état terminal de l'exécution satisfait Q
- Appelé de correction partielle car il spécifie uniquement le comportement prévu quand l'exécution termine

Correction partielle

Garantie que si le programme C termine, alors C satisfait la spécification S

- Par exemple,
  - $\{X = 1\}X := X + 1\{X = 2\}$ est valide
  - $\{X = 1\}X := X + 1\{X = 3\}$ n'est pas valide

Correction totale

Garantie que le programme C termine et C satisfait la spécification S

- Le triplet de correction totale $[P]\ C\ [Q]$ est valide si et seulement si
  - ↪ chaque fois que C est exécuté dans un état initial satisfaisant P
  - ↪ alors l'exécution doit terminer
  - ↪ et l'état terminal doit satisfaire Q
- Le triplet de correction totale suivant n'est pas valide:

$$[X = 1]\ \textit{while}\ X > 0\ \textit{do}\ X := X + 1\ [X = 0]$$

  - ↪ la boucle ne se termine jamais lorsqu'elle est exécutée à partir d'un état initial où X est positif
- Le triple de correction totale suivant est valide:

$$[X = 0]\ \textit{while}\ X > 0\ \textit{do}\ X := X + 1\ [X = 0]$$

  - ↪ la boucle se termine toujours immédiatement lorsqu'elle est exécutée à partir d'un état initial où X vaut zéro

Correction totale

- De manière informelle :

  Correction totale = terminaison + correction partielle
- Il est souvent plus facile de prouver séparément la correction partielle et la terminaison
- La terminaison est généralement simple à prouver, mais il existe des exemples où ce n'est pas le cas
- Prouver la terminaison nécessite la donnée d'une mesure décroissante pour chaque boucle (un variant), le plus souvent indiquée manuellement par l'utilisateur (voir plus loin dans le cours)
- Choix: Dépend du contexte : la correction partielle peut être suffisante!

Système formel de preuve pour Hoare-Logic

- ↪ Nous allons maintenant introduire un système de preuve par déduction pour les triplets à correction partielle grâce à Tony Hoare
- ↪ La logique consiste en un ensemble d'axiomes et de règles d'inférence pour déduire les conséquences à partir des prémisses

Système formel de preuve : Fondations

- Un système d'inférence (ou calcul logique) permet de déduire des formules à partir d'un ensemble de faits élémentaires (axiomes) et de faits déduits par des règles :

$$\frac{A_1 \ldots A_n}{A_{n+1}}$$

Sous les hypothèses (Assemptions) $A_1 \ldots A_n$
on peut déduire le but ou la conclusion A_{n+1}

↪ Une règle avec $n = 0$ est un fait élémentaire

↪ Les variables apparaissant dans les formules A_n peuvent être arbitrairement substituées

↪ La propriété $A[x \leftarrow e]$ signifie la propriété A dans laquelle toutes les occurences de la variable x ont été substituées par l'expression e

- Les variables d'une règle d'inférence peuvent être remplacées par une substitution
- La règle d'inférence substituée est appelée une instance (de cette règle)

Système formel de preuve : Fondations

Une preuve formelle (ou : dérivation) est un arbre avec des instances de règles comme nœuds

$$\dfrac{\dfrac{\dfrac{f(a,b)=a}{a=f(a,b)} \quad \dfrac{f(a,b)=a \quad f(f(a,b),b)=c}{f(a,b)=c}}{a=c} \quad \dfrac{}{g(a)=g(a)}}{g(a)=g(c)}$$

Les faits non élémentaires sur les feuilles sont les hypothèses globales, ici

- $f(a,b)=a$
- $f(f(a,b),b)=c$

Système formel de preuve pour les programmes
Hoare – Logic

- Nous avons les instructions/commandes (cmd) suivantes:

commande vide, sans effet	skip
affectation	$x := e \quad (x \in V)$
composition séquentielle	$c1; c2$
conditionnelle	*IF cond THEN* $c1$ *ELSE* $c2$
boucle	*WHILE cond DO c*

- où $c, c1, c2$ sont des cmds, V des variables, e une expression arithmétique et *cond* une expression booléenne

Système formel de preuve pour les programmes
Hoare – Logic

- Concept de base : Un triplet de Hoare composé de
  pré-condition P
  post-condition Q
  un programme cmd
- Le triple est écrit:

$$\vdash \{P\}\ cmd\ \{Q\} \text{ ou simplement } \{P\}\ cmd\ \{Q\}$$

P et Q sont des formules logiques sur les variables V, elles peuvent donc être considérées comme un ensemble d'états possibles

Intuitivement signifie:

Si un programme *cmd* démarre dans un état admis par P (la pécondition P est valide avant d'exécuter *cmd*), s'il se termine, le programme doit atteindre un état qui satisfait Q

Hoare – Logic: Sémantique axiomatique

$$\frac{}{\{P\}\ skip\ \{P\}}$$

La règle skip-rule

$$\frac{}{\{P[V \leftarrow E]\}\ V := E\ \{P\}}$$

La règle d'affectation

$$\frac{\{P \wedge cond\}\ c\ \{Q\} \qquad \{P \wedge \neg cond\}\ d\ \{Q\}}{\{P\}\ IF\ cond\ THEN\ c\ ELSE\ d\ \{Q\}}$$

La règle du conditionnel

$$\frac{\{P\}\ c\ \{R\} \qquad \{R\}\ d\ \{Q\}}{\{P\}\ c;\ d\ \{Q\}}$$

La règle de séquence

$$\frac{\{P \wedge cond\}\ c\ \{P\}}{\{P\}\ WHILE\ cond\ DO\ c\ \{P \wedge \neg cond\}}$$

La règle de boucle

$$\frac{P \implies P' \qquad \{P'\}\ cmd\ \{Q'\} \qquad Q' \implies Q}{\{P\}\ cmd\ \{Q\}}$$

La règle de conséquence

Hoare – Logic: La règle skip-rule

$$\boxed{\frac{}{\{P\}\ \mathit{skip}\ \{P\}}}$$

- L'axiome *skip – rule* exprime que toute assertion valide avant l'exécution de *skip* est également valide après l'exécution de *skip*
- Les états ne changent pas
- Par conséquent, les états valides restent valides

Example

$$\{X = 1\}\ \mathit{skip}\ \{X = 1\}$$

Hoare – Logic: La règle d'affectation

$$\boxed{\overline{\{P[V \leftarrow E]\}\ V := E\ \{P\}}}$$

- Ici, $P[V \leftarrow E]$ signifie l'assertion P avec l'expression E substituée à toutes les occurrences de la variable V
- Une façon de lire cette règle est la suivante : si vous voulez que l'instruction $V := E$ se termine dans un état qui satisfait l'assertion P, alors il suffit de commencer dans un état qui satisfait également P, mais où E est substitué à chaque occurrence de V
- 1st Exemple: $\{1 \leq x \wedge x \leq 10\}\ x := x + 2\ \{3 \leq x \wedge x \leq 12\}$

  Est-ce vraiment une instance de la règle d'affectation ? On calcule

  $$\begin{aligned}&(3 \leq x \wedge x \leq 12)\ [x \leftarrow x + 2]\\ &\equiv (3 \leq (x + 2) \wedge (x + 2) \leq 12)\\ &\equiv (1 \leq x \wedge x \leq 10)\end{aligned}$$

- 2nd Exemple: $\{\mathit{true}\}\ x := 2\ \{x = 2\}$

  Est-ce vraiment une instance de la règle d'affectation ? On calcule

  $$\begin{aligned}&(x = 2)\ [x \leftarrow 2]\\ &\equiv (2 = 2)\\ &\equiv \mathit{true}\ \text{(reflexivity)}\end{aligned}$$

Hoare – Logic: La règle du conditionnel

$$\frac{\{P \wedge cond\}\ c\ \{Q\} \qquad \{P \wedge \neg cond\}\ d\ \{Q\}}{\{P\}\ IF\ cond\ THEN\ c\ ELSE\ d\ \{Q\}}$$

- Certes, si la même assertion Q est valide après l'exécution de l'une ou l'autre des branches, alors elle est valide après l'ensemble du conditionnel
- Plus précis: Dans la branche 'then', nous savons que l'expression booléenne *cond* est évaluée à vrai, et dans la branche èlse', elle est évaluée à faux. Rendre ces informations disponibles dans les prémisses de la règle nous donne plus d'informations avec lesquelles travailler lors du raisonnement sur le comportement de c et d
- Par exemple, pour prouver $\{true\}\ if\ X \geq Y\ then\ Z := X\ else\ Z := Y\ \{Z = max(X, Y)\}$

$\hookrightarrow$ Il suffit de prouver ces 2 POs (Obligations de Preuve (ou Proof Obligations PO)):

$$\{true \wedge X \geq Y\}\ Z := X\ \{Z = max(X, Y)\} \quad \text{(PO1)}$$
$$\{true \wedge \neg(X \geq Y)\}\ Z := Y\ \{Z = max(X, Y)\} \quad \text{(PO2)}$$

POLYTECH PARIS-SACLAY

Hoare – Logic: La règle de séquence

$$\boxed{\frac{\{P\}\ c\ \{R\} \quad \{R\}\ d\ \{Q\}}{\{P\}\ c\ ;\ d\ \{Q\}}}$$

- Si la postcondition de c correspond à la précondition de d, nous pouvons dériver une spécification pour leur composition séquentielle
- Par exemple, si on a prouvé :

$\hookrightarrow$ $\{X = 1\}\ X := X + 1\ \{X = 2\}$
$\hookrightarrow$ $\{X = 2\}\ X := X + 1\ \{X = 3\}$
$\Longrightarrow$ On peut déduire que $\{X = 1\}\ X := X + 1; X := X + 1\ \{X = 3\}$

Hoare – Logic: La règle de la boucle while

$$\frac{\{P \wedge cond\}\ c\ \{P\}}{\{P\}\ WHILE\ cond\ DO\ c\ \{P \wedge \neg cond\}}$$

Justification informelle de la règle while

P	invariant initialement valide
$P \wedge cond$	condition supposée vraie
c	exécution du corps de la boucle
P	invariant rétabli
$P \wedge cond$	condition supposée vraie
c	exécution du corps de la boucle
P	invariant rétabli
...	n'importe quel nombre d'itérations
P	invariant rétabli
$P \wedge \neg cond$	boucle se termine lorsque la condition est fausse

- Le prédicat P est appelé un invariant
- Warning: Trouver un invariant de boucle adéquat est une difficulté majeure
- Nous reviendrons sur le problème de trouver des invariants de boucle

Hoare – Logic: La règle de consequence (modus-ponens)

$$\frac{P \implies P' \quad \{P'\}\ cmd\ \{Q'\} \quad Q' \implies Q}{\{P\}\ cmd\ \{Q\}}$$

$$\frac{P \implies P' \quad \{P'\}\ cmd\ \{Q\}}{\{P\}\ cmd\ \{Q\}}$$

$$\frac{\{P\}\ cmd\ \{Q'\} \quad Q' \implies Q}{\{P\}\ cmd\ \{Q\}}$$

- La règle de consequence permet de renforcer les preconditions et d'affaiblir les postconditions
- Si $A \implies B$, alors
  - ↪ la condition A est dite plus forte que B
  - ↪ la condition B est plus faible que A
- E.g., à partir de $\{X + 1 = 2\}\ X := X + 1\ \{X = 2\}$ on peut déduire $\{X = 1\}\ X := X + 1\ \{X = 2\}$

Hoare – Logic: Summary

- ↪ Hoare Logic est un formalisme permettant de raisonner sur le comportement des programmes en reliant leur état initial et leur état final
- ↪ Il utilise une logique d'assertion basée sur la logique du premier ordre pour raisonner sur les états des programmes et l'étend avec les triplets de Hoare pour raisonner sur les programmes
- ↪ Suggested reading:
  - C. A. R. Hoare. An axiomatic basis for computer programming. 1969
  - R. W. Floyd. Assigning meanings to programs. 1967

POLYTECH PARIS-SACLAY

Invariants de boucle & Correction totale

- Nous allons maintenant examiner les moyens de trouver des preuves, notamment :
  - ↪ règles dérivées & raisonnement à rebours (backwards reasoning)
  - ↪ trouver des invariants
  - ↪ façons d'annoter les programmes avant de les prouver

- Nous allons également examiner les règles de preuve pour une correction totale

Forward & backwards reasoning

- Les règles de preuve vues jusqu'à présent sont les mieux adaptées au raisonnement orienté vers l'avant (forward) où un arbre de preuve est construit à partir d'axiomes vers la spécification souhaitée
- Par exemple, considérons une preuve du triplet

  $\{X = a\}\ X := X + 1\ \{X = a + 1\}$

  en utilisant la règle d'affectation

$$\frac{}{\{P[V \leftarrow E]\}\ V := E\ \{P\}}$$

Backward-affectation

- Il est souvent plus naturel de travailler à rebours (Backward), en commençant par la racine de l'arbre de preuve et en générant de nouveaux sous-objectifs jusqu'à ce que toutes les feuilles soient des axiomes
- Nous pouvons dériver des règles mieux adaptées au raisonnement à rebours (Backward)
- Par exemple, nous pouvons dériver cette règle de

Backward-affectation :

$$\boxed{\frac{P \implies Q[V \leftarrow E]}{\{P\}\ V := E\ \{Q\}}}$$

Backwards affectation séquencée

- La règle de séquence peut déjà être appliquée en backward, mais nous oblige à deviner une assertion R :

$$\frac{\{P\}\ C1\ \{R\} \quad \{R\}\ C2\ \{Q\}}{\{P\}\ C1;\ C2\ \{Q\}}$$

- Dans le cas d'une commande séquencée avant une affectation, on peut éviter de devoir deviner R avec la règle d'affectation séquencée

Backward-Sequenced affectation :

$$\boxed{\frac{\{P\}\ C\ \{Q[V \leftarrow E]\}}{\{P\}\ C; V := E\ \{Q\}}}$$

Backward Loop

De la même manière, nous pouvons dériver une règle de raisonnement à rebours (Backward) pour les boucles en construisant en conséquence :
Backward-Loop :

$$\boxed{\frac{P \implies I \quad \{I \wedge B\}\ C\ \{I\} \quad I \wedge \neg B \implies Q}{\{P\}\ \textit{while}\ B\ \textit{Do}\ C\{Q\}}}$$

Cette règle nous oblige encore à deviner un invariant I

Notion d'invariant

Invariant

Un invariant est une propriété P des variables en début de boucle telle que si P est vérifiée à une itération, alors elle l'est à l'itération suivante

Méthodologie

1. Choisir et exprimer un invariant judicieux
   - Pas de méthode systématique
2. Démontrer qu'il est vérifié avant d'entrer dans la boucle
   - Utiliser les préconditions
3. Démontrer que s'il est vérifié au début d'une itération quelconque, il l'est aussi au début de l'itération suivante
   - Utiliser le corps de la boucle
4. Instancier l'invariant en sortie de boucle et en déduire une postcondition
   - Utiliser (la négation de) la condition du while

POLYTECH PARIS-SACLAY

Verifier la terminaison d'une boucle

Variant de boucle

Un variant est une expression v, à valeurs positives, qui decroit strictement à chaque itération d'une boucle et *assure sa terminaison*

Totale-Loop :

$$\frac{\forall\, V, [P \wedge B \wedge v = V]\ c\ [P \wedge v < V \wedge v \geq 0]}{[P]\ \textit{while}\ B\ \textit{do}\ c\ [P \wedge \neg B]}$$

- La boucle termine forcement, au bout d'au plus V iterations, où V est la valeur initiale du variant v
- Note : en cas de boucles imbriquees, on vérifie la terminaison de chaque boucle indépendamment des autres
- Un variant de boucle est une expression :
  - ↪ entière
  - ↪ positive
  - ↪ qui décroît strictement à chaque itération
- Variants usuels
  - i pour une boucle du type *for* $i = n$ *downto* 1
  - $n - i$ pour une boucle du type *for* $i = 1$ *to* n
  - $j - i$ pour deux variables i croissante et j décroissante
  - ...
  - mais pas de technique "systématique"

Règles de preuve (1/2)

$$\frac{P \implies Q}{\{P\}\ skip\ \{Q\}}$$ *Skip*

$$\frac{\{P\}\ C1\ \{R\} \quad \{R\}\ C2\ \{Q\}}{\{P\}\ C1;\ C2\ \{Q\}}$$ *Sequence*

$$\frac{P \implies Q[E \leftarrow V]}{\{P\}\ V := E\ \{Q\}}$$ *Affectation*

$$\frac{\{P\}\ C\ \{Q[E \leftarrow V]\}}{\{P\}\ C; V := E\ \{Q\}}$$ *Sequenced-Affectation*

$$\frac{\{P \wedge B\}C1\{Q\} \quad \{P \wedge \neg B\}C2\{Q\}}{\{P\}if\ B\ then\ C1\ else\ C2\{Q\}}$$ *Conditionnal*

Règles de preuve (2/2)

P invariant, $\dfrac{\{P \wedge B\}\ C\ \{P\}}{\{P\}\ \textit{while}\ B\ \textit{do}\ C\ \{P \wedge \neg B\}}$ While-1

$$\frac{P \implies I \quad \{I \wedge B\}C\{I\} \quad I \wedge \neg B \implies Q}{\{P\}\textit{while}\ B\ \textit{do}\ C\{Q\}}$$ *While-2*

$$\frac{\forall\ V, [P \wedge B \wedge v = V]\ C\ [P \wedge v < V \wedge v \geq 0]}{[P]\ \textit{while}\ B\ \textit{do}\ C\ [P \wedge \neg B]}$$ *While-Total correction*

$$\frac{P \implies P' \quad \{P'\}\ C\ \{Q'\} \quad Q' \implies Q}{\{P\}\ C\ \{Q\}}$$ *Consequence*